

# Unidad 7

## Programación de Funciones y Procedimientos almacenados

# Programación SQL

2

Contenido:

- ▶ Estructuras de Selección
- ▶ Estructuras de Iteración
- ▶ Manejadores (Handlers)
- ▶ Cursores

# Estructura de selección

3

## ► IF... ELSE

```
IF condicion_busqueda THEN lista_sentencias  
    [ELSEIF condicion_busqueda THEN lista_sentencias] ...  
    [ELSE lista_sentencias]  
END IF
```

# Estructura de selección

4

## ► CASE

- Podemos distinguir dos formas de uso: dependiendo del valor de una variable, o dependiendo del resultado de condiciones de búsqueda.
- En cualquier caso, si la evaluación es verdadera se ejecuta la lista de sentencias correspondiente. Si no, si no se encuentra ninguna coincidencia (o ninguna condición es verdadera) se ejecuta el bloque 'ELSE', este se podría poner vacío (begin end; ) si no se quiere tomar acción.

```
CASE valor
  WHEN valor1 THEN lista_sentencias
  [WHEN valor2 THEN lista_sentencias] ...
  [ELSE lista_sentencias]
END CASE
```

```
CASE
  WHEN condicion_busqueda THEN lista_sentencias
  [WHEN condicion_busqueda THEN lista_sentencias] ...
  [ELSE lista_sentencias]
END CASE
```

## ► Sentencia ITERATE

**ITERATE** *etiqueta*

- Se utiliza únicamente con las sentencias LOOP, REPEAT y WHILE, para indicar el punto desde el que se inicia la iteración ('etiqueta')

## ► Sentencia LEAVE

**LEAVE** *etiqueta*

- Se utiliza para salir del bloque de instrucciones identificado con 'etiqueta':
  - bloque begin..end
  - bucles (loop, repeat, while).
- Si la etiqueta es para el bloque más externo del procedimiento entonces se forzaría la salida del procedimiento.

# Estructura de iteración

6

## ► LOOP

```
[etiqueta:] LOOP  
    lista_sentencias;  
END LOOP [etiqueta]
```

- Se utiliza en combinación con las sentencias 'ITERATE' y 'LEAVE'. Se evalúa la condición que corresponda para determinar si seguir iterando (iterate) o si por el contrario es necesario salir de la iteración (leave)

```
CREATE PROCEDURE factorial(IN num INT,  
                           OUT fact INT)  
BEGIN  
    DECLARE i int;  
    set i = num;  
    set fact = num;  
    bucle: LOOP  
        SET i = i-1;  
        SET fact = fact*i;  
        IF i > 1 THEN  
            ITERATE bucle;  
        END IF;  
        LEAVE bucle;  
    END LOOP bucle;  
END
```

# Estructura de iteración

7

## ► REPEAT

```
[etiqueta:] REPEAT  
    lista_sentencias  
UNTIL condicion_busqueda  
END REPEAT [etiqueta]
```

- En un bucle con 'repeat' se garantiza que el bloque de instrucciones se ejecuta al menos una vez.
- Se iterará hasta que se cumpla la condición en 'until'

```
CREATE FUNCTION ffactorial(num INT)  
RETURNS int  
BEGIN  
    DECLARE i,fact int;  
    set i = num;  
    set fact = num;  
    REPEAT  
        SET i = i-1;  
        SET fact = fact*i;  
    UNTIL i <= 1  
    END REPEAT;  
    return fact;  
END  
//
```

# Estructura de iteración

8

## ► WHILE

```
[etiqueta:] WHILE  
condicion_busqueda DO  
    lista_sentencias  
END WHILE [etiqueta]
```

- Se itera el bloque de sentencias mientras sea verdadera la condición de búsqueda.

```
CREATE FUNCTION ffactorialw(num INT)  
RETURNS int  
BEGIN  
    DECLARE i,fact int;  
    set i = num;  
    set fact = num;  
    WHILE i > 1 DO  
        SET i = i-1;  
        SET fact = fact*i;  
    END WHILE;  
    RETURN fact;  
END  
//
```



# Manejadores / Excepciones

9

- ▶ **HANDLER:** Es el mecanismo utilizado para el control de excepciones durante la ejecución de los procedimientos almacenados.
- ▶ **Tipos:**
  - ▶ **CONTINUE:** Cuando se produce una excepción, el código **continúa** con la siguiente instrucción después del error.
  - ▶ **EXIT:** Cuando se produce una excepción, el código **sale** del bloque en el que está definido el HANDLER

# Manejadores / Excepciones

10

## ► SINTAXIS

```
DECLARE {CONTINUE | EXIT} HANDLER
FOR condicion [, condicion] ...
Sentencia(s)
```

Donde

```
condicion: {
    mysql_error_code
    | SQLSTATE [VALUE] valor_sqlstate
    | nombre_condicion
    | SQLWARNING
    | NOT FOUND
    | SQLEXCEPTION
}
```

- *Mysql\_error\_code* : código de error mysql
- SQLSTATE [*VALUE*]: código sqlstate
- *Nombre\_condicion*: nombre de una condición previamente definida con DECLARE.. CONDITION
- SQLWARNING: SQLSTATE empezando por '01'
- NOT FOUND : SQLSTATE empezando por '02'
- SQLEXCEPTION: SQLSTATE que no empiezan por '00','01', '02'.

# Manejadores / Excepciones

11

- ▶ Es posible declarar más de un manejador en un mismo procedimiento.
- ▶ Cuando hay más de un manejador declarado:
  - ▶ Se evalúan todos los manejadores declarados
  - ▶ En caso de error se utiliza el más específico que coincida con el error producido.
  - ▶ No es relevante el orden en el que se declaran los manejadores

- ▶ Los **cursores** se definen dentro de procedimientos almacenados y nos permiten recorrer el conjunto de registros del resultado de una 'select':
  - ▶ Son de solo lectura
  - ▶ Se pueden recorrer únicamente en un sentido, sin saltar filas.
  - ▶ Se declaran antes que los manejadores y después de variables y condiciones.

- Pasos en el uso de un cursor:

1. **Declarar** el cursor

**DECLARE** nombre\_cursor CURSOR FOR sentencia\_select;

- Pueden declararse varios cursores en una rutina, pero el nombre debe ser único en cada bloque.

2. **Abrir** el cursor

**OPEN** nombre\_cursor;

3. **Recorrer** el cursor

**FETCH** [[NEXT] FROM] nombre\_cursor INTO variable [, variable] ...

- Con FETCH se avanza (con un puntero interno) al siguiente registro del cursor abierto.
- Cuando se llega al final de los registros devueltos en la 'select' se da la condición SQLSTATE 02000 (NOT FOUND) y es habitual utilizar un manejador para detectar esta situación y así, de forma controlada, continuar con la ejecución del programa.

4. **Cerrar** el cursor

**CLOSE** nombre\_cursor;